

Lecture 4

Introduction to spatial modelling – spatial data types

Sara Martino, Dept. of Mathematical Science, NTNU,

Janine Illian,

Jafet Belmont, University of Glasgow

May 7, 2026

- Why spatial modelling?
- Why is spatial modelling computationally expensive?
- Different data types:
 - Modelling in discrete space – areal data
 - Modelling in continuous space – geo-referenced data
 - Modelling continuous space – spatial point process data
- representation of data structures in R

Spatial modelling

Many natural processes take place in space.

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

$\rightsquigarrow$ complex spatial models required

challenges:

- often inaccessible to practitioners (literature aimed at statisticians)

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

$\rightsquigarrow$ complex spatial models required

challenges:

- often inaccessible to practitioners (literature aimed at statisticians)
- methodology not always linked to applications

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

$\rightsquigarrow$ complex spatial models required

challenges:

- often inaccessible to practitioners (literature aimed at statisticians)
- methodology not always linked to applications
- models may be too simple to reflect real-life data

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

$\rightsquigarrow$ complex spatial models required

challenges:

- often inaccessible to practitioners (literature aimed at statisticians)
- methodology not always linked to applications
- models may be too simple to reflect real-life data
- difficult to apply without expertise in spatial stats and computational statistics

Spatial modelling

Many natural processes take place in space.

Large amounts of data collected in space:

increased resolution $\rightsquigarrow$ large, complex datasets

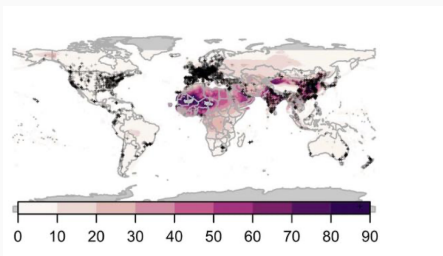
$\rightsquigarrow$ complex spatial models required

challenges:

- often inaccessible to practitioners (literature aimed at statisticians)
- methodology not always linked to applications
- models may be too simple to reflect real-life data
- difficult to apply without expertise in spatial stats and computational statistics

We will see that `inlabru` can help...

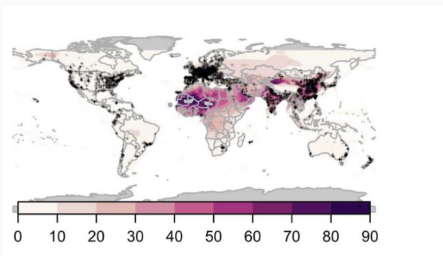
Example: Global PM 2.5



Exposure to air pollution; particulate matter $< 2.5\mu m$ (PM 2.5)

- linked to poor health outcomes; responsible for 3 million deaths worldwide each year
- observations potentially **not independent**
- sparsely measured
- heterogeneous spatial coverage

Example: Global PM 2.5



Exposure to air pollution; particulate matter $< 2.5\mu m$ (PM 2.5)

- linked to poor health outcomes; responsible for 3 million deaths worldwide each year
- observations potentially **not independent**
- sparsely measured
- heterogeneous spatial coverage

spatial dependence + complex observation processes

Spatial modelling: why necessary?

- standard models assume independent observations
- spatial data are (typically) not independent
- two nearby observations are similar $\rightarrow$ not providing independent information

spatial dependence – autocorrelation

Ignoring spatial (or any other, e.g. temporal) dependence:

- pretending we have as much independent pieces of information as the number observations
- pretending we have more information than is actually available

Ignoring spatial (or any other, e.g. temporal) dependence:

- pretending we have as much independent pieces of information as the number observations
- pretending we have more information than is actually available

~> spuriously tight confidence intervals

~> wrong inference and wrong conclusions

spatial dependence – autocorrelation

Ignoring spatial (or any other, e.g. temporal) dependence:

- pretending we have as much independent pieces of information as the number observations
 - pretending we have more information than is actually available
- ~> spuriously tight confidence intervals
- ~> wrong inference and wrong conclusions

Spatial models include specific **components** to explicitly represent dependence.

Ignoring spatial (or any other, e.g. temporal) dependence:

- pretending we have as much independent pieces of information as the number observations
- pretending we have more information than is actually available

~> spuriously tight confidence intervals

~> wrong inference and wrong conclusions

Spatial models include specific **components** to explicitly represent dependence.

There are many different ways of “being dependent”, hence these **components** and their properties vary with each application.

dependence relationships need to be represented in the model –

dependence relationships need to be represented in the model –
which observation is related to which other observations?

dependence relationships need to be represented in the model –
which observation is related to which other observations?

- usually represented by a matrix
- some matrix operations (e.g. inversion) are computationally expensive
- in the past: often MCMC $\rightsquigarrow$ very slow

dependence relationships need to be represented in the model – which observation is related to which other observations?

- usually represented by a matrix
- some matrix operations (e.g. inversion) are computationally expensive
- in the past: often MCMC $\rightsquigarrow$ very slow
- INLA (and hence `inlabru`) much faster...

Spatial modelling: aims

aims vary among different studies, e.g.:

- describing spatial patterns and structures
- understanding spatial patterns and structures
- linking data observed in space to covariates while accounting for spatial autocorrelation
- predicting into unobserved locations

Spatial modelling: aims

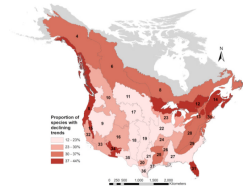
aims vary among different studies, e.g.:

- describing spatial patterns and structures
- understanding spatial patterns and structures
- linking data observed in space to covariates while accounting for spatial autocorrelation
- predicting into unobserved locations

aims also vary with different types of spatial data...

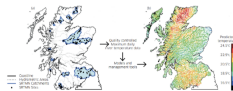
Types of spatial data

Areal data



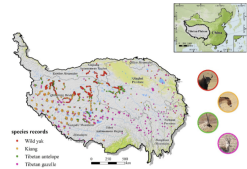
Map of bird conservation regions (BCRs) showing the proportion of bird species within each region showing a declining trend

Geostatistical data



Scotland river temperature monitoring network

Point pattern data



Occurrence records of four ungulate species in Tibet,

Discrete space:

- Data on a spatial grid (areal data)

Continuous space:

- Geostatistical (geo-referenced) data
- Spatial point patterns (point-referenced data)

Model components are used to reflect spatial dependence structures in discrete and continuous space.

Discrete space: areal data

- Data on a (regular or irregular) spatial grid
- Examples: number of individuals in a region, average rainfall in a province
- Originally point or geostatistical data; gridded for practical reasons

Observed response(s): Measurement associated with each grid cell or areal unit

Discrete space: areal data

- Data on a (regular or irregular) spatial grid
- Examples: number of individuals in a region, average rainfall in a province
- Originally point or geostatistical data; gridded for practical reasons

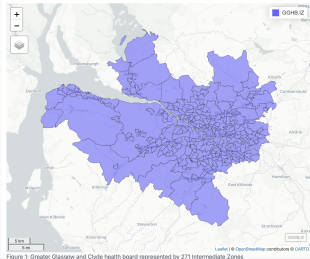
Observed response(s): Measurement associated with each grid cell or areal unit

here: spatial structure represented by Gauss Markov random field (GMRF)

the data we will use

respiratory hospitalisations in Glasgow:

- areal data on respiratory hospitalisations
- 271 Intermediate Zones (IZ) in Greater Glasgow and Clyde health board
- covariates concern air pollution concentration and socio-economic deprivation in Scotland



Standardized Mortality Ratios (SMR)

Response here:

disease risk, estimated using **Standardized Mortality Ratios (SMR)**: for spatial areal unit i SMR is the ratio between the observed (Y_i) and expected (E_i) number of cases:

$$SMR_i = \frac{Y_i}{E_i}$$

Standardized Mortality Ratios (SMR)

Response here:

disease risk, estimated using **Standardized Mortality Ratios (SMR)**: for spatial areal unit i SMR is the ratio between the observed (Y_i) and expected (E_i) number of cases:

$$SMR_i = \frac{Y_i}{E_i}$$

$SMR > 1$: more observed cases than expected

⇒ **high risk area**

$SMR < 1$, fewer observed cases than expected

⇒ **low risk area**

Continuous space: geostatistical data

- phenomenon continuous in space
- measured at a finite set of locations
- examples: nutrient levels in soil, salinity in the sea

Observed response(s): measurements at given locations in continuous space

Continuous space: geostatistical data

- phenomenon continuous in space
- measured at a finite set of locations
- examples: nutrient levels in soil, salinity in the sea

Observed response(s): measurements at given locations in continuous space

here: spatial structure represented by Gaussian random field;

Continuous space: geostatistical data

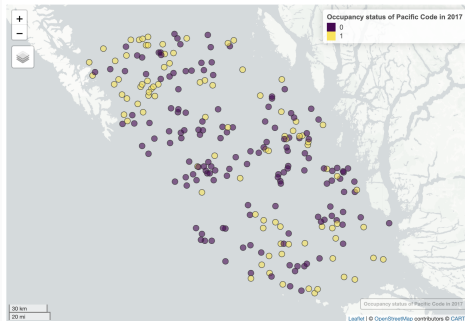
- phenomenon continuous in space
- measured at a finite set of locations
- examples: nutrient levels in soil, salinity in the sea

Observed response(s): measurements at given locations in continuous space

here: spatial structure represented by Gaussian random field; approximated by a continuously indexed Gauss Markov random field

the data we will use

- Pacific Cod (*Gadus macrocephalus*)
- trawl survey in Queen Charlotte Sound



Continuous space: spatial point patterns

- Locations of objects/events in space (typically 2D)
- Examples: tree locations, animal groups, earthquakes

Observed response(s): x, y coordinates (sometimes also additional measurements; “marks”)

modelled by a random variable, a spatial point process characterised by intensity $\lambda(s)$, $s \in R$

Continuous space: spatial point patterns

- Locations of objects/events in space (typically 2D)
- Examples: tree locations, animal groups, earthquakes

Observed response(s): x, y coordinates (sometimes also additional measurements; “marks”)

modelled by a random variable, a spatial point process characterised by intensity $\lambda(s)$, $s \in R$

here: intensity is a Gaussian random field;

Continuous space: spatial point patterns

- Locations of objects/events in space (typically 2D)
- Examples: tree locations, animal groups, earthquakes

Observed response(s): x, y coordinates (sometimes also additional measurements; “marks”)

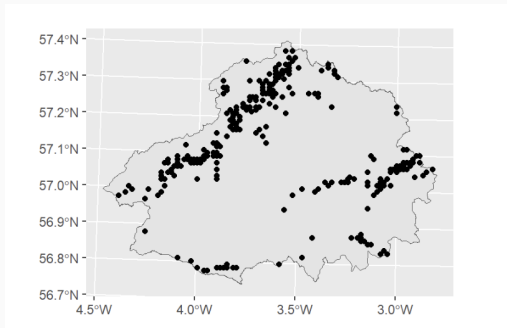
modelled by a random variable, a spatial point process characterised by intensity $\lambda(s)$, $s \in R$

here: intensity is a Gaussian random field;

approximated by a continuously indexed Gauss Markov random field

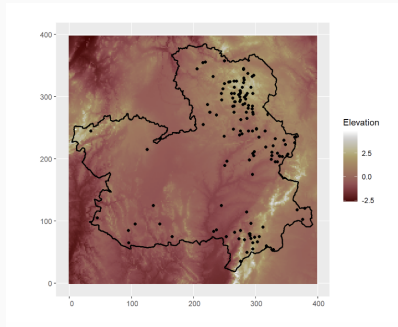
the data we will use

location of sightings of the ringlet butterfly in Scotland's Cairngorms National Park



the data we will use

locations of forest fires in the Castilla-La Mancha region of Spain between 1998 and 2007



Different spatial data structures...

- different types of spatial data structure – **modelling**:
 - modelled with different statistical models

Different spatial data structures...

- different types of spatial data structure – **modelling:**
 - modelled with different statistical models
 - used to answer different questions
 - should not be mixed up

Different spatial data structures...

- different types of spatial data structure – **modelling:**
 - modelled with different statistical models
 - used to answer different questions
 - should not be mixed up
- different types of spatial data structure – **computation:**
 - spatial structures can all be approximated by a GMRF

Different spatial data structures...

- different types of spatial data structure – **modelling:**
 - modelled with different statistical models
 - used to answer different questions
 - should not be mixed up
- different types of spatial data structure – **computation:**
 - spatial structures can all be approximated by a GMRF
 $\rightsquigarrow$ the SPDE approach

Different spatial data structures...

- different types of spatial data structure – **modelling:**
 - modelled with different statistical models
 - used to answer different questions
 - should not be mixed up
- different types of spatial data structure – **computation:**
 - spatial structures can all be approximated by a GMRF
 $\rightsquigarrow$ the SPDE approach
 - GMRF can be efficiently fitted with INLA/inlabru
 - different types of models can be efficiently fitted with INLA/inlabru
 $\rightsquigarrow$ within the same framework

Different spatial data structures...

- different types of spatial data structure – **modelling:**
 - modelled with different statistical models
 - used to answer different questions
 - should not be mixed up
 - different types of spatial data structure – **computation:**
 - spatial structures can all be approximated by a GMRF
 $\rightsquigarrow$ the SPDE approach
 - GMRF can be efficiently fitted with INLA/inlabru
 - different types of models can be efficiently fitted with INLA/inlabru
- $\rightsquigarrow$ within the same framework
- different data structures can be jointly modelled

Point patterns vs. geostatistical data

Point patterns:

- Data format: x,y coordinates
- Optional: marks
- Aim: modelling pattern formed by locations; locations are random

Geostatistical data:

- Data format: x,y coordinates
- Measurements mandatory
- Aim: model continuous process based on measurements; locations of measurements are not random/have been chosen

take-home message

- all spatial models discussed as part of this course are special cases of latent Gaussian models
- different spatial terms are needed for different spatial data structures
- the SPDE approach unifies approximation as all data structures are spatially referenced in continuous space
- `inlabru` efficiently fits these models

What happens next?

Practical 4: introduction to spatial data wrangling and manipulation

- exploring tools for visualization and wrangling
- for different spatial data structures

Spatial data structures - the `sf` package

"Simple Features" for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data

Spatial data structures - the `sf` package

"Simple Features" for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data
- represents geometries (POINT, LINESTRING, POLYGON, etc.) as list-columns in data frames

Spatial data structures - the `sf` package

"Simple Features" for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data
- represents geometries (POINT, LINESTRING, POLYGON, etc.) as list-columns in data frames
- stores both **attributes** and **geometry** in a single object – a spatially-enabled data frame

Spatial data structures - the `sf` package

"Simple Features" for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data
- represents geometries (`POINT`, `LINESTRING`, `POLYGON`, etc.) as list-columns in data frames
- stores both **attributes** and **geometry** in a single object – a spatially-enabled data frame
- integrates seamlessly with `tidyverse` (e.g., `dplyr`, `ggplot2`)

Spatial data structures - the `sf` package

"Simple Features" for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data
- represents geometries (`POINT`, `LINESTRING`, `POLYGON`, etc.) as list-columns in data frames
- stores both **attributes** and **geometry** in a single object – a spatially-enabled data frame
- integrates seamlessly with `tidyverse` (e.g., `dplyr`, `ggplot2`)
- uses fast, modern spatial operations

Spatial data structures - the `sf` package

”Simple Features” for R (`sf`): modern, efficient framework for working with spatial data

- implements the **Simple Features** – `sf` standard for spatial vector data
- represents geometries (`POINT`, `LINESTRING`, `POLYGON`, etc.) as list-columns in data frames
- stores both **attributes** and **geometry** in a single object – a spatially-enabled data frame
- integrates seamlessly with `tidyverse` (e.g., `dplyr`, `ggplot2`)
- uses fast, modern spatial operations
- replaces `sp`

terra: designed for working with spatial **raster** and **vector** data

Key Features

- handles large raster datasets efficiently
- provides a unified framework for raster and vector data structures
- supports common GIS operations:
 - reprojection, resampling, cropping, masking
 - raster and map algebra
 - zonal statistics, extraction, distance calculations
- integrates with `sf` for combined raster–vector workflows
- supports numerous spatial data formats via the GDAL library.
- successor to `raster` package; faster, more memory-efficient tools

Comparing sf and terra in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

`terra`

- Handles **vector data**:
points, lines, polygons.

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

`terra`

- Handles **vector data**:
points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

`terra`

- Handles **vector data**:
points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

`terra`

- Handles **vector data**:
points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

`terra`

- Handles **vector data**:
points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.
- Efficient memory management and parallel processing.

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.
- Efficient memory management and parallel processing.
- Can read, write, and process large spatial

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.
- Efficient memory management and parallel processing.
- Can read, write, and process large spatial

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.
- Efficient memory management and parallel processing.
- Can read, write, and process large spatial

Comparing `sf` and `terra` in R

Both `sf` and `terra` are modern R packages for working with spatial data, but they are designed for different types of spatial objects.

`sf` (*simple features*)

- Handles **vector data**: points, lines, polygons.
- Follows the Simple Features standard (ISO 19125).
- Stores geometry and attributes together in a data frame.
- Integrates seamlessly with the tidyverse (`dplyr`, `ggplot2`).

`terra`

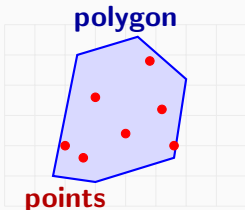
- Handles **raster data**: continuous gridded surfaces (e.g., elevation, temperature).
- Replaces the older raster package.
- Efficient memory management and parallel processing.
- Can read, write, and process large spatial

vector vs. raster data:

sf vs. terra

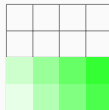
sf: vector data

- discrete geometric objects: points, lines, polygons
- store exact shapes and boundaries
- used for administrative areas, roads, sample sites



terra: raster data

- continuous grid of cells (pixels)
- each cell stores a numeric value (e.g., elevation, temperature)
- used for spatially explicit covariates that have values everywhere in space

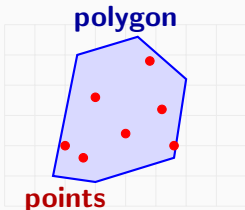


vector vs. raster data:

sf vs. terra

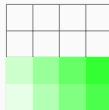
sf: vector data

- discrete geometric objects: points, lines, polygons
- store exact shapes and boundaries
- used for administrative areas, roads, sample sites



terra: raster data

- continuous grid of cells (pixels)
- each cell stores a numeric value (e.g., elevation, temperature)
- used for spatially explicit covariates that have values everywhere in space



further slides

sf vs. sp

sf is a modern replacement of the sp package

old: sp

modern: sf

- introduced in early 2000s

sf vs. sp

sf is a modern replacement of the sp package

old: sp

modern: sf

- introduced in early 2000s
- complex S4 class system

sf vs. sp

sf is a modern replacement of the sp package

old: sp

modern: sf

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes

sf vs. sp

sf is a modern replacement of the sp package

old: sp

modern: sf

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2

sf vs. sp

sf is a modern replacement of the sp package

old: sp

modern: sf

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure
- geometry stored as a list-column (sfc)

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure
- geometry stored as a list-column (sfc)
- works seamlessly with dplyr, ggplot2, tidyr

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure
- geometry stored as a list-column (sfc)
- works seamlessly with dplyr, ggplot2, tidyr
- automatic CRS handling and conversions

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure
- geometry stored as a list-column (sfc)
- works seamlessly with dplyr, ggplot2, tidyr
- automatic CRS handling and conversions

sf vs. sp

sf is a modern replacement of the sp package

old: sp

- introduced in early 2000s
- complex S4 class system
- separate objects for geometry and attributes
- limited integration with dplyr/ggplot2
- manual handling of coordinate reference systems

modern: sf

- introduced in 2016, built on the Simple Features standard
- uses tidyverse-friendly data frame structure
- geometry stored as a list-column (sfc)
- works seamlessly with dplyr, ggplot2, tidyr
- automatic CRS handling and conversions

sf simplifies workflows, speeds up computation, and integrates